



Informatics II

Exercise 2

Grgic Anto

Schedule (Corrected)

Date	Topic
27. Feb	C environment
06. Mar	Ex1(Introduction to C, basic sorting)
13. Mar	Ex2 (Recursion)
20. Mar	Ex3 (Complexity, Correctness)
27. Mar	Ex4 (Divide and Conquer, Recurrence)
03. Apr	NO LAB (Please attend another class) – Spring break
17. Apr	Ex6 (Pointers, Linked Lists)
24. Apr	Ex7(ADTs, Stacks, Queues)
01. May	NO LAB (Please attend another class) – Labor day
08. May	Ex9 (Hash Tables)
15. May	Ex10 (Dynamic Programming)
22. May	Ex11 (Graphs)
29. May	Recap
04. Jun	Exam at Messe 16:00-19:00

Recursion

```
int factorial(int n) {  
  
    if (n == 0) return 1; // base case  
  
    return n * factorial(n - 1); // body + recursive call  
}
```

↓ CALLING

factorial (4) → calls factorial(3) CALL

factorial (3) → calls factorial(2) CALL

factorial (2) → calls factorial(1) CALL

factorial (1) → calls factorial(0) CALL

factorial (0) → 1 BASE CASE

↑ RETURNING

factorial (1) → $1 \times 1 = 1$ RETURN

factorial (2) → $2 \times 1 = 2$ RETURN

factorial (3) → $3 \times 2 = 6$ RETURN

factorial (4) → $4 \times 6 = 24$ FINAL

Base case: A simple value is returned. The calling “loop” stops and the returning “loop” starts.

Body: Some kind of operation get applied (Often to the recursive call’s returning value).

Recursive call: The function calls itself with a parameter that depends on the parameter of the current iteration.

Recursion

Why use recursion?

- Sometimes is easier to implement than iterative counterpart.

```
int fib_recursive(int n) {  
    if (n <= 1)  
        return n;  
    return fib_recursive(n - 1) + fib_recursive(n - 2);  
}
```

VS

```
int fib_iterative(int n) {  
    if (n <= 1)  
        return n;  
    int a = 0, b = 1;  
    for (int i = 2; i <= n; i++) {  
        int temp = a + b;  
        a = b;  
        b = temp;  
    }  
    return b;  
}
```

- When using divide and conquer algorithms or when solving DPs. (More of it in Ex.4 and Ex.10)

Why not use recursion?

- Requires memory space.
- Can perform same calculation multiple times (Without memoization).
- Harder to debug.

Task 1.

```
main() {  
    str = "racecar"  
    n = strlen(str)  
    res = false  
    if  $n > 0$  then  
        res = RecursiveAlgo(str, 0, n - 1)  
    printf("%s", res?"true":"false")  
}  
  
RecursiveAlgo(str, s, e){  
    if  $s == e$  then  
        return true  
    if  $str[s] != str[e]$  then  
        return false  
    if  $s < e + 1$  then  
        return RecursiveAlgo(str, s + 1, e - 1)  
    return true  
}
```

-What is s, what is e and what is str when we call RecursiveAlgo() the 1st time?

-What are the possible returns of RecursiveAlgo()?

-What are the base cases?

-What are the cases we do the recursive call?

-How does str, s and e change when we make the recursive call?

Task 2.

```
void printRec(int n)
{
    if (n == 0)
        return;

    printf("%d", n%2);

    printRec(n/2);
}
```

- Compute is by hand.
- Remember the call stack.

Task 3.

- Use recursion:

- Base case?

- Body?

- Recursion step?

Task 3.

[See animation on Olat:](#)



Task 3.

Understanding the solution:

```
void Hanoi(int n, int A, int B, int C) {  
    if (n == 0) return;          //BASE CASE  
  
    Hanoi(n-1, A, C, B);        //Take top n-1 disks from A, put them on C, use B as auxiliary  
  
    p[B] = p[B] * 10 + p[A] % 10; //Take top disk from A (after you took n - 1 on C), put it in B  
    p[A] = p[A] / 10;  
  
    M++;  
    print_peg();  
  
    Hanoi(n-1, C, B, A);        //Take top n-1 disks from C (Where you put the previous n-1 from A), put them in B, use A as auxiliary  
}
```

Task 4.

```
void draw_pyramid(const int A[], int n, int level) {  
    // base case: nothing to print / reduce  
    if (n < 1) {  
        return;  
    }  
  
    // recursive case: build reduced array of length n-1, then recurse  
    int t[n - 1];  
    for (int i = 0; i < n - 1; i++) {  
        t[i] = A[i] + A[i + 1];  
    }  
  
    draw_pyramid(t, n - 1, level + 1);  
  
    // print AFTER recursion so that top of pyramid appears first (top-down)  
    print_level(A, n, level);  
}
```

```
int n = (int)(sizeof(A) / sizeof(A[0]));
```

This is how you find out the size on an array in the main (it doesn't work in functions)

Thank you for your attention.

**If you have any question write in the Olat forum or
come up to me.**